

Базы данных: SQL и NoSQL

Проектирование модели данных, нормализация, ключи и связи, SQL для аналитика, транзакции и изоляция, индексы и планы, масштабирование, семейства NoSQL.

Редакция от 06.08.2026

Содержание

1. Проектирование модели данных
 - 1.1 Три уровня
 - 1.2 Как искать сущности
 - 1.3 Атрибуты, о которых забывают
 - 1.4 Историчность
2. Реляционная модель
 - 2.1 Ключи
 - 2.2 Связи
 - 2.3 Нормализация
 - 2.4 Типы данных и ограничения
3. SQL для аналитика
 - 3.1 Что нужно уметь писать
 - 3.2 Группы команд
4. Транзакции
 - 4.1 ACID
 - 4.2 Аномалии и уровни изоляции
 - 4.3 Конкурентный доступ
5. Производительность
 - 5.1 Индексы
 - 5.2 План запроса
 - 5.3 Типичные проблемы
6. Масштабирование
7. NoSQL
 - 7.1 Семейства
 - 7.2 CAP и BASE
 - 7.3 Когда выбирать NoSQL
8. Хранилища и аналитика
9. Вопросы для самопроверки

1. Проектирование модели данных

Модель данных это то, что переживает интерфейсы, фреймворки и команды. Экран переделывают за неделю, ошибку в модели данных исправляют миграцией на миллионах строк с простым и риском потери. Поэтому аналитику дороже ошибиться в чём угодно, кроме модели.

1.1 Три уровня

Концептуальная модель. Сущности и связи, без атрибутов и типов. Язык предметной области, а не базы. Помещается на одну страницу и обсуждается с заказчиком.

Логическая модель. Атрибуты, ключи, обязательность, нормализация, разрешение связей многие-ко-многим. Без привязки к конкретной СУБД.

Физическая модель. Имена, точные типы, индексы, ограничения, партиции, права. Привязана к СУБД.

Аналитик обязан строить первые два уровня и уметь читать третий. Разговор с заказчиком ведётся на концептуальном уровне, разговор с разработчиком на логическом.

1.2 Как искать сущности

Рабочий приём: выписать существительные из описания процесса и проверить каждое тремя вопросами.

1. У него есть свойства, которые нужно хранить?
2. Экземпляры различимы между собой и нужно их различать?
3. У него есть жизненный цикл или история?

Существительное, прошедшее проверку, становится сущностью. Не прошедшее становится атрибутом другой сущности или исчезает.

Второй приём: спросить заказчика «как вы это находите» и «что вы про это помните». Если сотрудник ищет заявку по номеру, номер обязан быть атрибутом с уникальностью и индексом. Если помнит, кто её принимал, нужна связь с пользователем.

1.3 Атрибуты, о которых забывают

Перечень, который стоит прогонять по каждой сущности:

- Кто создал и когда, кто изменил последним и когда.
- Признак удаления, если удаление мягкое, и кто удалил.
- Версия записи для оптимистичной блокировки.
- Внешний идентификатор для интеграций, если сущность приходит извне.

- Период действия, если запись действительна не всегда.
- Порядок сортировки, если пользователь может менять порядок вручную.

1.4 Историчность

Отдельный вопрос, который обязан быть задан на старте: нужна ли история изменений, и в каком виде.

Подход	Как устроен	Когда применять
Без истории	Только текущее состояние	Справочники, которые меняются редко и без последствий
Журнал изменений	Отдельная таблица с записями «что, кто, когда, было, стало»	Аудит, прослеживаемость действий
Версионирование строк	Каждая версия отдельной строкой с периодом действия	Нужно восстановить состояние на дату
Журнал событий	Хранятся события, состояние вычисляется	Финансы, сложная предметная область

Вопрос заказчику, который вскрывает потребность: «нужно ли вам знать, как выглядела эта запись полгода назад, и зачем». Ответ «на всякий случай» означает журнал изменений; ответ «мы обязаны предъявить проверяющему» означает версионирование с периодами.

Общий журнал аудита обычно дешевле в сопровождении, чем десяток отдельных таблиц истории на каждую сущность: у него одна схема, один механизм записи и один экран просмотра.

2. Реляционная модель

2.1 Ключи

Первичный ключ однозначно определяет строку. Требования: уникален, не пуст, не меняется.

Естественный или суррогатный. Естественный ключ берётся из предметной области (номер паспорта, ИНН). Суррогатный не имеет смысла вне системы (автоинкремент, UUID). Практика склоняется к суррогатным: естественные ключи имеют свойство меняться, повторяться и оказываться не такими уникальными, как обещал заказчик.

Автоинкремент или UUID. Автоинкремент компактен, упорядочен, хорош для индексов, но раскрывает объём данных и мешает слиянию баз. UUID генерируется на любой стороне, не раскрывает объём, но занимает больше места и хуже ложится в индекс, если не использовать упорядоченные варианты.

Составной ключ из нескольких колонок. Уместен в связующих таблицах.

Внешний ключ ссылается на первичный ключ другой таблицы. Поведение при удалении родителя указывается явно: каскадное удаление, обнуление ссылки, запрет удаления. Это решение аналитика, а не разработчика: удаление организации не должно молча уносить историю её заявок.

2.2 Связи

Тип	Как реализуется	Пример
Один к одному	Внешний ключ с уникальностью либо общий первичный ключ	Пользователь и его расширенный профиль
Один ко многим	Внешний ключ на стороне «многих»	Организация и её сотрудники
Многие ко многим	Ассоциативная таблица с двумя внешними ключами	Заявка и согласующие
Иерархия	Ссылка на себя либо материализованный путь	Подразделения

Про ассоциативную таблицу нужно помнить: у связи бывают собственные атрибуты. Связь «заявка и согласующий» несёт решение, комментарий, время голосования и признак обязательности. Как только у связи появляются атрибуты, она становится полноценной сущностью со своей историей.

2.3 Нормализация

1НФ. Атомарность значений: в ячейке одно значение, нет повторяющихся групп колонок вроде `phone1` , `phone2` , `phone3` .

2НФ. Каждый неключевой атрибут зависит от всего составного ключа, а не от его части.

3НФ. Нет транзитивных зависимостей: неключевой атрибут не зависит от другого неключевого. Классический пример нарушения: в таблице сотрудников хранятся `department_id` и `department_name` ; название зависит от идентификатора отдела, а не от сотрудника.

Практический критерий 3НФ: каждый факт хранится в одном месте. Если название организации изменилось и его нужно править в трёх таблицах, модель не нормализована.

Денормализация это осознанное нарушение ради скорости чтения. Допустима, когда измерена проблема, а не предполагается. Обязательно фиксируется в постановке вместе с ответом на вопрос, как поддерживается согласованность продублированных данных.

Особый случай, который денормализацией не является: сохранение исторического значения. Наименование организации в закрытой заявке хранят копией не ради скорости, а потому что документ должен показывать состояние на момент оформления. Это разные факты, и путать их нельзя.

2.4 Типы данных и ограничения

Решения, которые принимает аналитик:

- **Строки.** Ограничение длины ставится из предметной области, а не наугад. Поле для текста произвольной длины (комментарий, описание с форматированием) должно иметь тип текста без ограничения: ограничение в 255 символов рано или поздно даст ошибку сохранения на реальных данных.
- **Числа.** Денежные суммы хранятся в типе с фиксированной точностью или в минимальных единицах целым числом. Числа с плавающей точкой для денег дают расхождения в копейках, которые потом ищут неделями.
- **Дата и время.** Хранить в UTC, отображать в местном поясе. Отдельно решить, что значит «дата без времени» и как она ведёт себя на границе суток.
- **Перечисления.** Отдельная таблица-справочник или ограничение на значения. Хранение статуса произвольной строкой на русском языке удобно на старте и болезненно потом: аналитика по статусам ломается от опечатки, а внешней системе приходится сопоставлять литералы.
- **NULL.** Означает «значение неизвестно или неприменимо», а не ноль и не пустую строку. Для каждого поля решается явно, допустим ли NULL и чем он отличается от пустого значения по смыслу.
- **Ограничения целостности.** Уникальность, проверка значения, обязательность. Правило, выраженное ограничением базы, невозможно нарушить; правило, выраженное только в коде, будет нарушено при первой же массовой загрузке данных.

3. SQL для аналитика

3.1 Что нужно уметь писать

Выборка с соединениями. Понимать разницу типов соединения и уметь объяснить, почему выбран именно этот.

Соединение	Что возвращает
INNER JOIN	Только строки, у которых есть пара в обеих таблицах
LEFT JOIN	Все строки левой таблицы, пары из правой или NULL
RIGHT JOIN	Симметрично левому
FULL JOIN	Все строки обеих таблиц
CROSS JOIN	Декартово произведение
SELF JOIN	Соединение таблицы с собой, для иерархий

Классическая ловушка: условие на правую таблицу в LEFT JOIN, вынесенное в WHERE вместо ON, превращает соединение во внутреннее и молча выбрасывает строки.

Агрегаты и группировка. COUNT, SUM, AVG, MIN, MAX, группировка, фильтрация групп через HAVING. Разница между WHERE и HAVING: первый фильтрует строки до группировки, второй группы после.

Подзапросы и обобщённые табличные выражения. CTE читаются лучше вложенных подзапросов и позволяют строить рекурсивные запросы для иерархий.

Оконные функции. То, что отличает уверенное владение от базового: ROW_NUMBER, RANK, LAG, LEAD, SUM с OVER. Позволяют считать в разрезе, не теряя строки.

Пример задачи уровня middle: доля отказов по организациям за период, с числом заявок и рангом.

```
WITH stat AS (  
    SELECT  
        o.name AS organization,  
        COUNT(*) AS total,  
        COUNT(*) FILTER (WHERE a.status = 'Отказано') AS refused  
    FROM applications a  
    JOIN organizations o ON o.id = a.organization_id  
    WHERE a.sending_datetime >= '2026-01-01'  
        AND a.sending_datetime < '2026-07-01'  
    GROUP BY o.name  
    HAVING COUNT(*) >= 10
```

```

)
SELECT
    organization,
    total,
    refused,
    ROUND(refused * 100.0 / total, 1) AS refusal_rate,
    RANK() OVER (ORDER BY refused * 1.0 / total DESC) AS rank
FROM stat
ORDER BY refusal_rate DESC;

```

Что проверяет такая задача: умение считать долю без деления на ноль, понимание разницы WHERE и HAVING, работа с полуинтервалом дат вместо BETWEEN (важно, когда в поле есть время), приведение типов при делении целых.

3.2 Группы команд

Группа	Команды	Назначение
DDL	CREATE, ALTER, DROP, TRUNCATE	Определение структуры
DML	SELECT, INSERT, UPDATE, DELETE	Работа с данными
DCL	GRANT, REVOKE	Права доступа
TCL	COMMIT, ROLLBACK, SAVEPOINT	Управление транзакциями

4. Транзакции

4.1 ACID

Атомарность. Транзакция выполняется целиком либо не выполняется вовсе.

Согласованность. База переходит из одного корректного состояния в другое, ограничения целостности соблюдены.

Изолированность. Параллельные транзакции не мешают друг другу; степень невмешательства задаётся уровнем изоляции.

Долговечность. Подтверждённые изменения переживают отказ.

Практическое следствие для постановки: любая операция, меняющая несколько таблиц, должна быть транзакцией. Формулировка «создаётся заявка, её вложения и запись в истории» подразумевает, что при сбое на третьем шаге не остаётся ни заявки, ни вложений.

4.2 Аномалии и уровни изоляции

Аномалия	Суть
Грязное чтение	Прочитаны неподтверждённые изменения другой транзакции
Неповторяющееся чтение	Повторное чтение той же строки даёт другое значение
Фантомное чтение	Повторный запрос по условию возвращает новые строки
Потерянное обновление	Два обновления по прочитанному значению, одно затирает другое

Уровень	Грязное	Неповторяющееся	Фантомы
Read Uncommitted	возможно	возможно	возможно
Read Committed	нет	возможно	возможно
Repeatable Read	нет	нет	возможно
Serializable	нет	нет	нет

Чем строже уровень, тем выше цена в блокировках и откатах. По умолчанию в большинстве систем используется Read Committed.

4.3 Конкурентный доступ

Вопрос, который аналитик обязан задать по любой форме редактирования: что произойдёт, если два человека откроют одну запись и сохранят одновременно.

Оптимистичная блокировка. У записи есть версия; при сохранении проверяется, что версия не изменилась. Если изменилась, пользователь получает сообщение о том, что данные устарели. Подходит, когда конфликты редки.

Пессимистичная блокировка. Запись блокируется на время редактирования. Подходит, когда конфликты часты и дороги, требует механизма снятия зависших блокировок.

Ответ «мы об этом не думали» означает, что в системе будет молчаливая потеря данных, которую заметят через полгода.

5. Производительность

5.1 Индексы

Индекс ускоряет поиск и замедляет запись. Каждый индекс это дополнительная структура, которую нужно обновлять при каждой вставке и изменении.

Тип	Назначение
В-дерево	Основной тип: равенство, диапазоны, сортировка
Хеш	Только точное совпадение
Полнотекстовый и GIN	Поиск по словам и по фрагментам, по массивам и JSON
Пространственный	Географические данные
Частичный	Индексируется только часть строк по условию
Покрывающий	Содержит все нужные запросу колонки, чтение таблицы не требуется

Составной индекс работает слева направо: индекс по (организация, дата) помогает запросу по организации и по дате, но не помогает запросу только по дате.

Когда индекс не работает: функция или преобразование над колонкой в условии, начало поиска с произвольного места строки без соответствующего типа индекса, низкая избирательность (колонка с двумя значениями на миллион строк), приведение типов в условии.

Вопрос, который аналитик обязан закрыть в постановке: по каким полям пользователь ищет и сортирует. Список полей поиска это и есть техническое задание на индексы.

5.2 План запроса

Аналитику полезно уметь читать план хотя бы поверхностно: полный проход по таблице против поиска по индексу, тип соединения, оценка числа строк против фактического. Резкое расхождение оценки и факта означает устаревшую статистику. Умение сказать «здесь полный проход по таблице на два миллиона строк» на разборе инцидента ценится.

5.3 Типичные проблемы

Запрос в цикле. Список из ста элементов, для каждого отдельный запрос за связанными данными. Решается пакетной выборкой или предварительной загрузкой. Проявляется как «на тестовых данных быстро, на реальных виснет».

Пагинация большими смещениями. Переход на тысячную страницу заставляет базу отбросить тысячи строк. Решается пагинацией по ключу.

Подсчёт общего числа записей на каждой странице списка. На больших таблицах дороже самой выборки. Решается приблизительной оценкой или отказом от показа точного числа.

Отчёты по боевой базе. Тяжёлый аналитический запрос конкурирует с работой пользователей. Решается репликой для чтения или отдельным хранилищем.

6. Масштабирование

Приём	Что делает	Когда применять
Вертикальное	Больше ресурсов одному серверу	Первое и простейшее решение
Репликация	Копии базы для чтения	Много чтения, мало записи
Партиционирование	Одна таблица разбита на части по ключу	Большие журналы, разбиение по датам
Шардирование	Данные разнесены по разным серверам	Не помещается на один сервер
Кэширование	Часто читаемое хранится в быстром хранилище	Повторяющиеся запросы

Партиционирование по дате особенно полезно для журналов: старые разделы удаляются одной операцией вместо удаления миллионов строк, а запросы за период читают только нужные разделы.

7. NoSQL

7.1 Семейства

Тип	Модель	Представители	Когда уместен
Ключ-значение	Ключ и произвольное значение	Redis, Memcached	Кэш, сессии, счётчики, блокировки
Документный	Документы JSON с вложенностью	MongoDB	Гибкая схема, документ читается целиком
Колоночный	Строки с семействами колонок	Cassandra, HBase	Огромные объёмы записи, временные ряды
Графовый	Узлы и рёбра	Neo4j	Связи важнее сущностей: рекомендации, маршруты, мошенничество
Поисковый	Обратный индекс	Elasticsearch, OpenSearch	Полнотекстовый поиск, анализ журналов
Временные ряды	Метки времени и значения	TimescaleDB, InfluxDB	Метрики, показания датчиков

7.2 CAP и BASE

Теорема CAP: при разделении сети распределённая система выбирает между согласованностью и доступностью. Формулировка «выберите два из трёх» неточна: разделение сети не выбирают, оно случается, выбор происходит между двумя оставшимися свойствами.

BASE как противоположность ACID: доступность в приоритете, состояние мягкое, согласованность достигается в конечном счёте. Практический смысл для постановки: после записи данные могут быть видны не сразу. Если интерфейс показывает список сразу после создания записи, пользователь может не увидеть только что созданное. Это поведение нужно либо предотвратить, либо описать явно.

7.3 Когда выбирать NoSQL

Честный ответ, который стоит дать на собеседовании: в корпоративных системах реляционная база остаётся выбором по умолчанию, а NoSQL добавляется для конкретной задачи рядом с ней. Кэш в Redis, поиск в Elasticsearch, журналы во временных рядах при основной базе PostgreSQL это типичная и здоровая архитектура.

Признаки, что задача просит документное хранилище: структура записи заметно различается от объекта к объекту, данные читаются и пишутся целым документом, связей между документами почти нет.

Признаки, что задача просит графовую базу: главный вопрос к данным звучит как «кто с кем связан и через сколько шагов».

Плохой довод в пользу NoSQL: «схема ещё не устоялась». Отсутствие схемы в базе не означает её отсутствия: схема переезжает в код приложения, где её никто не контролирует.

8. Хранилища и аналитика

Различие, которое стоит понимать, даже если не работаешь с аналитикой напрямую.

Признак	OLTP	OLAP
Назначение	Ежедневные операции	Анализ и отчётность
Запросы	Много коротких, точечных	Мало тяжёлых, по большим объёмам
Модель	Нормализованная	Звезда или снежинка: факты и измерения
Оптимизация	Быстрая запись и точечное чтение	Быстрая агрегация

Терминология, которая встречается в вакансиях: **хранилище данных** это структурированные данные, подготовленные для анализа; **озеро данных** это сырые данные в исходном виде; **витрина** это срез хранилища под конкретное подразделение; **ETL** это извлечение, преобразование и загрузка; **ELT** это тот же процесс с преобразованием уже внутри хранилища.

Для системного аналитика достаточно понимать разницу и уметь сформулировать требование к выгрузке: какие данные, с какой глубиной истории, с какой периодичностью, в каком разрезе, где допустима задержка.

9. Вопросы для самопроверки

Если на каждый вопрос находится развёрнутый ответ с примером, блок освоен.

1. Чем первичный ключ отличается от уникального индекса.
2. Почему связь многие-ко-многим нельзя выразить без третьей таблицы.
3. Что произойдёт при удалении организации, у которой есть заявки, при каждом из вариантов поведения внешнего ключа.
4. В чём разница между WHERE и HAVING и почему это не вопрос вкуса.
5. Почему условие на правую таблицу в WHERE ломает LEFT JOIN.
6. Когда нарушение третьей нормальной формы оправдано.
7. Чем хранение исторического значения отличается от денормализации.
8. Какие аномалии допускает уровень изоляции Read Committed и чем это грозит.
9. Как два пользователя могут потерять данные при одновременном редактировании и как это предотвратить.
10. Почему составной индекс по (a, b) не помогает запросу только по b.
11. Почему пагинация по смещению деградирует и чем её заменить.
12. Чем документная база отличается от реляционной по цене изменения структуры.
13. Что означает согласованность в конечном счёте для интерфейса пользователя.
14. Почему деньги нельзя хранить в числах с плавающей точкой.
15. Зачем хранить время в UTC, если все пользователи в одном часовом поясе.